



**SHAHEED HIMAYUN MUZZAMIL MEMORIAL
GOVERNMENT DEGREE COLLEGE ANANTNAG**

Department of Computer Applications

**Project Report (e-yantra)
Line Following Robot with PID Control**

Batch 2022 (NEP)

Aqib Javid Bhat	65274-ANG-2022
Mohd Numan Asgar	65301-ANG-2022
Danish Ellahi	65277-ANG-2022
Ubaid Mushtaq Mir	65291-ANG-2022
Wasiq Tariq	63848-ANG-2021
Ifham Mushtaq	65292-ANG-2022

**Under the Supervision of
Dr. Tawseef Ahmed Teli
Asst. Prof. Department of Computer Applications**

Contents

Abstract	2
1. Introduction.....	2
1.1 Background and Motivation	2
1.2 Problem Statement.....	2
1.3 Scope and Applications	3
2. Methodology	3
2.1 Working Principle.....	3
2.2 Hardware Platform (AlphaBot2-Ar).....	4
2.2.1 AlphaBot2-Base.....	5
2.2.1 AlphaBot2-Ar Adapter Board.....	6
2.3 Software and Algorithms Used	7
2.3.1 Programming Language and IDE	7
2.3.2 Libraries Used.....	7
2.3.3 Algorithm and Pseudocode	8
2.3.4 Flowchart.....	10
2.3.5 Code	10
3. Implementation.....	13
3.1 Build and Assembly.....	13
3.2 Challenges Faced and How They Were Resolved.....	13
3.3 Assumptions.....	14
4. Testing and Results.....	14
4.1 Tests Performed	14
4.2 Observations and Outcomes	15
5. Conclusion and Future Scope.....	15
6. GitHub Repository.....	16

Abstract

Autonomous mobile robots have become an integral part of modern industrial and service environments, enabling efficient, hands-free movement along predefined routes. Line-following robots, a specific class of these systems, rely on infrared (IR) sensors to detect visual cues such as black lines on white surfaces, reducing the need for human supervision and improving operational efficiency. Earlier implementations using Bang-Bang motor control, while simple, often resulted in abrupt speed changes, oscillations, and inefficient energy consumption due to sudden corrections amplified by sensor noise and uneven surfaces. To address these limitations, the project implemented a PID-controlled line-following robot on the Waveshare AlphaBot2-Ar platform. This system employs a five-sensor IR array to continuously detect the line position, and the PID controller adjusts the differential motor speeds to maintain precise alignment with the line center. The design incorporates a calibration phase where sensors are exposed to both black and white surfaces, allowing accurate normalization and minimizing tracking errors. Through iterative experimentation, PID gains were optimized to $K_p=1/5$, $K_i=1/10000$, and $K_d=30$, achieving smooth, stable navigation. Testing under various conditions, including straight paths, curved routes, and moderate lighting variations, demonstrated consistent, oscillation-free performance, validating the effectiveness of PID control compared to traditional Bang-Bang approaches. This project not only addresses stability and accuracy in line following but also provides a scalable framework for educational and industrial applications in autonomous robotics.

1. Introduction

1.1 Background and Motivation

The preceding project on a Line Following localization Robot (available at <https://github.com/Misgar-Numan/Line-Following-Localization-Robot>) used “Bang-Bang” motor control to initiate steering adjustments. Although effective for simple tasks, this method caused significant oscillations from sudden speed variations, worsened by sensor noise and surface inconsistencies, resulting in inefficient energy consumption and path deviations. This constraint inspired the transition to PID control, a feedback mechanism rooted in classical control theory, which handles error magnitude (Proportional), accumulation (Integral), and rate (Derivative) for refined adjustments.

1.2 Problem Statement

The primary challenge in line-following robotics is achieving stable, oscillation-free tracking on varied surfaces and curves. The Bang-Bang motor control’s discrete actions amplify errors, resulting in oscillations and path deviations that compromise accuracy and component longevity. This necessitates a closed-loop feedback system that continuously senses line deviation, computes corrections using PID, and executes differential steering, all while ensuring computational efficiency on resource-constrained microcontrollers. This demands sensor integration for accurate line tracking, algorithmic tuning for stability, and hardware calibration for platform variability, ultimately yielding precise, oscillation-free navigation suitable for real-world deployment. The AlphaBot2-Ar platform was selected after initial DIY failures due to uneven IR spacing and power inconsistencies which caused erratic readings, leading to deviation from actual path. This evolution not only addresses oscillatory flaws but also introduces adaptive calibration, fostering a scalable

framework for advanced robotics curricula.

1.3 Scope and Applications

The scope encompasses designing, implementing, and validating a PID-controlled line follower on AlphaBot2-Ar platform, emphasizing straight-line stability, curve handling, and calibration for black line on flat white surfaces under consistent indoor lighting. It excludes dynamic obstacles or full localization, focusing solely on control theory which includes sensor normalization, weighted positioning, PID tuning, differential drive for speeds.

The **Line Following Robot** has wide-ranging applications in both industrial and service sectors. In modern manufacturing and warehouse environments, it serves as the foundation for **Automated Guided Vehicles (AGVs)** that transport materials, tools, and products along predefined routes, thereby improving productivity, safety, and accuracy. These robots are extensively used in **Assembly Line Automation**, where they carry components between workstations without human supervision, ensuring a continuous production process. In the **healthcare sector**, line-following robots are employed in hospitals to deliver medicines, test samples, and patient records, maintaining hygiene and efficiency by reducing direct human contact. They are also adopted in **airports, metro stations, and logistics hubs** to handle luggage and goods automatically along marked floor paths, supporting the concept of **smart transportation systems**. Beyond industrial and commercial uses, line-following robots have significant **educational and research value**, providing an ideal platform for students to understand embedded systems, sensor calibration, control algorithms, and robotics programming. Their principles are further adapted in **domestic service robots**, such as robotic vacuum cleaners and food delivery bots, which rely on path tracking for smooth movement. In **theme parks, museums, and exhibitions**, these robots can function as interactive tour guides, following paths while providing audio or visual information to visitors. Additionally, they find potential use in **agriculture and rescue operations**, where robots can follow designated tracks for sowing, spraying, or navigating hazardous environments. Overall, the Line Following Robot demonstrates the fundamental concept of autonomous navigation and serves as a stepping stone toward the development of advanced intelligent systems used in **smart industries, automation, and autonomous vehicles**.

2. Methodology

2.1 Working Principle

The Line-Following Robot with PID Control employs a closed-loop system where five down-facing ITR20001/T reflective infrared photoelectric sensors detect line position, feeding into a PID controller that adjusts left and right motor speeds for centered tracking. Upon power-on, the Arduino configures the sensors and motors, and runs a 10-second calibration phase, where each sensor is exposed to black and white contrasts, recording min and max reflectance per sensor for normalization. The `readLine()` function then yields a weighted position reading (0-4000) for the line, where “2000” means the line is centered under the robot, “0” means far left, and “4000” far right. The setpoint is chosen as “2000” to keep the line in the center. The error e is computed as:

$$e = \text{position} - 2000.$$

The PID algorithm then computes the correction value using three components:

- **Proportional (P):** $P = \text{error}$, giving an output directly proportional to the current error.

- **Integral (I):** $I = \sum \text{error}$, accumulating past errors to eliminate steady-state bias (e.g., uneven sensor readings).
- **Derivative (D):** $D = \text{error} - \text{previous_error}$, predicting future error trend to dampen overshoot.

The PID output is computed as

$$PID_{output} = Kp * P + Ki * I + Kd * D.$$

The gains Kp, Ki, Kd are tuned experimentally. The PID output is then mapped to a differential motor command

$$\text{Motor_speed} = \text{Base_speed} \pm PID_{output}.$$

This continuous adjustment yields smooth movement instead of on/off turns.

2.2 Hardware Platform (AlphaBot2-Ar)



Figure 1: AlphaBot2-Ar

The AlphaBot2-Ar kit developed by Waveshare was used as the hardware platform for implementing the Line Following Robot. It is a compact, integrated mobile robotics platform built for rapid prototyping and control system testing with snap-fit components and no soldering required. The AlphaBot2-Ar kit includes a chassis (AlphaBot2-Base) and an Adapter Board (AlphaBot2-Ar) for Arduino UNO integration.

2.2.1 AlphaBot2-Base

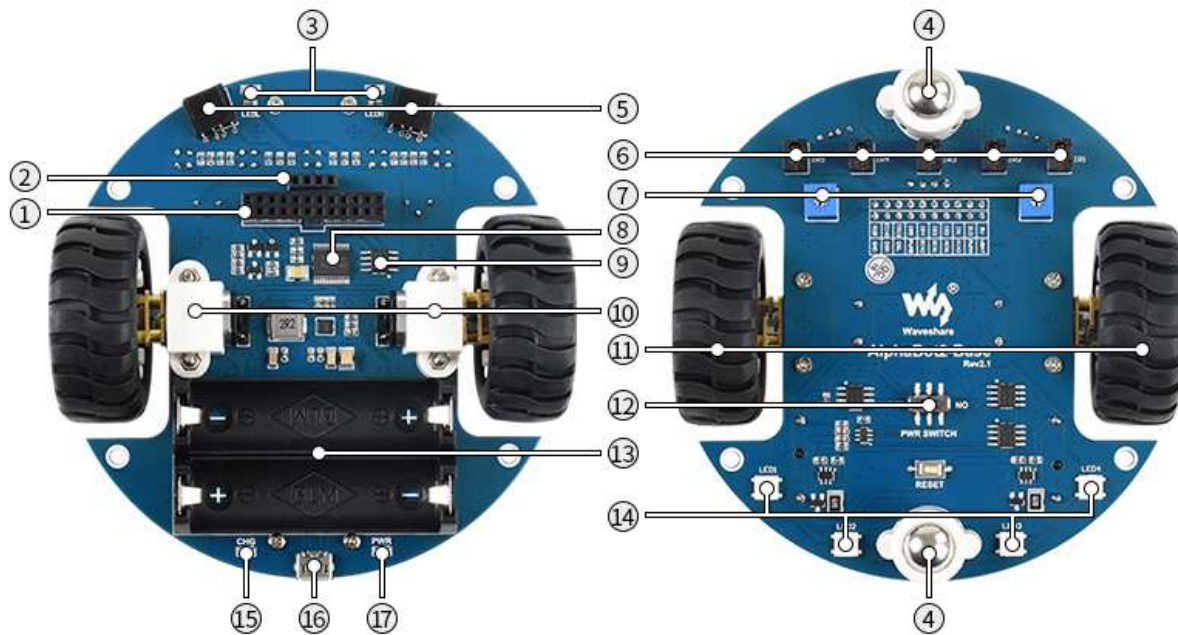


Figure 2: AlphaBot2-Base

The key components of the AlphaBot2-Base include the following:

1. **AlphaBot2 Control Interface** connects the AlphaBot2-Base to the controller adapter board (e.g., Arduino, Raspberry Pi, Jetson Nano). It provides the communication and power interface between the controller and the base. Through this connector, control signals (PWM, direction, sensor inputs) are transferred between the microcontroller and the base hardware.
2. **Ultrasonic Module Interface** connects an HC-SR04 or similar Ultrasonic sensor. It allows the robot to perform obstacle detection or distance measurement by sending and receiving ultrasonic pulses. This port provides VCC, GND, TRIG and ECHO pins for the sensor.
3. **Obstacle-Avoiding Indicators** are visual indicators (LEDs) that show the detection state of obstacle sensors. When an obstacle is detected by the front IR sensors (ST188), the corresponding LED turns ON. This helps during debugging and sensor calibration.
4. **Omni-Directional Wheel** is a small castor wheel that allows the robot to move freely in any direction while the main drive wheels steer and propel. It reduces friction and stabilizes turns.
5. **ST188 – Reflective Infrared Photoelectric Sensor (Obstacle Avoiding)** detect nearby obstacles using infrared reflection. It emits IR light and detects reflected light from nearby objects. When an obstacle is within a certain range, the sensor outputs a digital signal to the microcontroller.
6. **ITR20001/T – Reflective Infrared Photoelectric Sensor (Line Tracking)** is used for line following. It consists of an IR LED and a phototransistor. It detects reflected IR light from surface, black surfaces reflect less (low output) and white surfaces reflect more (high output). The sensors are arranged at the front underside of the robot to sense the line's position for the PID control.

7. **Potentiometer** is used for adjusting the sensitivity or detection range of the obstacle-avoiding sensors (ST188). It works by rotating the knob that changes the voltage threshold of the LM393 comparator, altering the distance at which the obstacle sensors trigger.
8. **TB6612FNG Dual H-bridge Motor Driver** is an integrated circuit that powers two N20 micro-gear motors. It accepts PWM and direction signals from the microcontroller to control each motor's speed and direction. It can handle up to 13.5v and ~1.2A per channel.
9. **LM393 Voltage Comparator** processes analog signals from IR sensors. It compares the sensor voltage with a reference voltage (set by potentiometer) to produce a digital high/low signal indicating detection (used for obstacle sensors).
10. **N20 Micro Gear Motors** are compact DC motors with built-in gear reduction (1:30 reduction rate, 6 V/600 RPM) for torque and controllable speed. These provide the driving power to the robot's wheels.
11. **Rubber Wheels** are high-friction wheels of diameter 42 mm and width 19 mm, ensuring smooth and stable traction on various surfaces.
12. **Power Switch** turns the robot's power system ON or OFF, controlling overall power delivery to motors and electronics.
13. **Battery Holder** supports two 14500 lithium-ion cells (3.7 V each). It supplies ~7.4 V to the motor driver and regulators. The base includes built-in charging and voltage regulation circuits.
14. **WS2812B True-Color RGB LEDs** are individually addressable RGB LEDs controlled by the microcontroller for status, decoration, or debugging.
15. **Battery Charging Indicator** is an LED indicator for charging status. It Lights up during charging and turns off (or changes color) when charging is complete.
16. **Charging Port** is a micro-USB port connected to the charging circuit. It allows recharging using any standard 5 V USB power source.
17. **Power Indicator** is an LED that lights up when power is supplied to the base board, confirming operational status.

2.2.1 AlphaBot2-Ar Adapter Board

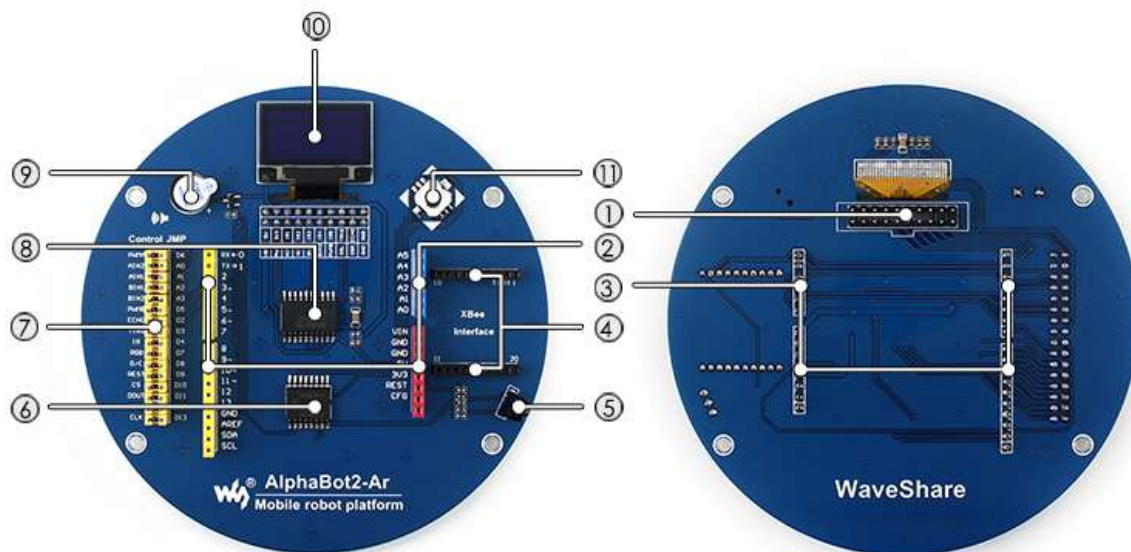


Figure 3: AlphaBot2-Ar Adapter Board

The key components of the AlphaBot2-Ar adapter board include the following:

1. **AlphaBot2 Control Interface** is a connector that links the AlphaBot2-Ar adapter board to the AlphaBot2-Base, transferring power, motor control signals, and sensor data between the two boards.
2. **Arduino Expansion Header** allows additional shields or external modules to be connected directly to the Arduino controller for extended functionality.
3. **Arduino Interface** is a socket designed to mount an Arduino-compatible controller (such as Arduino UNO) onto the AlphaBot2-Ar board, enabling it to control the AlphaBot2-Base.
4. **XBee Connector** is an interface for attaching an XBee or dual-mode Bluetooth module, allowing wireless communication and remote control of the robot.
5. **IR Receiver** receives infrared signals from an IR remote control, enabling basic remote operation like start, stop, or direction control.
6. **PCF8574 (I/O Expander)** is an I2C-based 8-bit input/output expander that provides additional digital I/O pins for the Arduino, used for controlling sensors, LEDs, or other peripherals.
7. **Arduino Peripheral Jumpers** are configuration jumpers that allow customization of signal routing between the Arduino pins and peripheral modules on the AlphaBot2-Ar board.
8. **TLC1543 (10-bit ADC)** is a 10-bit analog-to-digital converter used to read analog signals from multiple sensors (e.g., line-tracking IR sensors) and send digital data to the Arduino.
9. **Buzzer** provides audio feedback for events such as startup tones, alerts, or line-loss warnings.
10. **0.96-inch OLED (SSD1306 Driver, 128×64 Resolution)** is a miniature display used to show system information such as sensor readings, PID values, or robot status in real-time.
11. **Joystick** is used for manual input, navigation through OLED menus, or controlling the robot's movement manually during testing.

2.3 Software and Algorithms Used

This section outlines the programming tools, logic, and flow that control the behavior of the robot.

2.3.1 Programming Language and IDE

The entire robot is programmed using the Arduino IDE, which is a user-friendly open-source platform for writing and uploading code to the Arduino Uno. The language used is C/C++, specifically tailored for embedded systems programming. The Arduino IDE offers built-in functions for controlling digital and analog pins, reading sensor data, and communicating with external devices, which made it easy to implement the logic required for this project.

2.3.2 Libraries Used

Libraries in Arduino are pre-written code that simplify programming by allowing users to interact with hardware using simple functions instead of dealing with complex code. To help build the robot and make the development process smoother, the TRSensors.h library was used.

The TRSensors.h library is provided by Polulu's QTRSensors library via Arduino Library Manager

to read the analog reflectance sensors. This library handles reading and calibrating up to an 8-channel QTR array, returning calibrated values and the computed line position. This reduced code complexity and improved sensor response time, ensuring smoother real-time control.

2.3.3 Algorithm and Pseudocode

The algorithm for the Line-Following Robot with PID control enables autonomous navigation along a black line on a white surface using infrared sensor array feedback. The robot continuously adjusts its path to maintain alignment with the line's centerline. It uses a five-sensor IR array to detect line position for PID corrections and executes differential steering at each cycle to maintain accuracy. The system ensures stable motion without oscillation, outperforming traditional bang-bang control through continuous proportional adjustments.

The PID gains ($K_p = 1/5$, $K_i = 1/10000$, $K_d = 30$), maximum motor speed (70), and centerline reference (2000) are both platform-dependent and environment-dependent. While the current values are optimized for the TRSensor array and motor configuration, they should be adjusted incrementally based on the robot's motor characteristics, weight, wheel diameter, and sensor calibration, as well as environmental factors such as line width, surface contrast, and ambient lighting. Proper tuning ensures smooth, oscillation-free tracking before deployment.

2.3.3.1 Algorithm

Objective: Implement autonomous line following using PID control with five IR sensors to maintain alignment with the line centerline while minimizing oscillation.

1. Initialization Phase
 - i. Configure motor control pins (PWM and direction pins for left and right motors).
 - ii. Configure TRSensor array with 5 IR reflectance sensors.
 - iii. Calibrate sensors over 400 iterations (~10 seconds) to establish min/max reflectance values.
 - iv. Initialize PID control variables: proportional, integral, last_proportional.
 - v. Set PID constants: $K_p = 1/5$, $K_i = 1/10000$, $K_d = 30$.
 - vi. Define control parameters: maximum = 70 (PWM saturation limit).
 - vii. Set centerline reference: 2000 (midpoint of 0-4000 sensor range).
 - viii. Display calibration values on serial monitor.
2. Sensor reading Phase
 - i. Read all five IR sensors simultaneously.
 - ii. Compute weighted line position (0-4000 scale) using TRSensor library.
 - iii. Store position value in position variable.
3. PID Error Calculation Phase
 - i. Calculate positional error: $\text{proportional} = \text{position} - 2000$.
 - ii. Compute derivative term: $\text{derivative} = \text{proportional} - \text{last_proportional}$.
 - iii. Accumulate integral term: $\text{integral} += \text{proportional}$.
 - iv. Update last_proportional for next iteration.
4. PID Control Output Phase
 - i. Compute PID output: $\text{power_difference} = (\text{proportional}/5) + (\text{integral}/10000) + (\text{derivative} \times 30)$.
 - ii. Clamp output to [-maximum, +maximum] range to prevent saturation
 - iii. Output power_difference to serial monitor.
5. Motor Speed Calculation Phase
 - i. Positive power_difference (line to right): Reduce right motor speed.

- ii. Negative power_difference (line to left): Reduce left motor speed.
 - iii. Apply PWM speeds to left and right motors.
6. Loop Continuation
Repeat sensor reading, PID calculation, and motor control phases continuously.

2.3.3.2 Pseudocode

BEGIN Line-Following Robot PID Control

 Initialize()

 Configure motor pins (PWMA=6, AIN1=A1, AIN2=A0, PWMB=5, BIN1=A2, BIN2=A3)

 Initialize TRSensor array[NUM_SENSORS] with NUM_SENSORS $\leftarrow$ 5

 Set motor directions: AIN1=LOW, AIN2=HIGH, BIN1=LOW, BIN2=HIGH

 Calibrate sensors: FOR i $\leftarrow$ 0 TO 400 DO trs.calibrate() END FOR

 Print calibratedMin[0..4] and calibratedMax[0..4] to serial

 Initialize: last_proportional $\leftarrow$ 0, integral $\leftarrow$ 0

 Set maximum $\leftarrow$ 70, centerline $\leftarrow$ 2000

 END

 WHILE TRUE

 // READ SENSORS

 position $\leftarrow$ trs.readLine(sensorValues) // Returns 0-4000 weighted position

 // PID ERROR CALCULATION

 proportional $\leftarrow$ position - centerline

 derivative $\leftarrow$ proportional - last_proportional

 integral $\leftarrow$ integral + proportional

 last_proportional $\leftarrow$ proportional

 // PID CONTROL OUTPUT

 power_difference $\leftarrow$ (proportional / 5) + (integral / 10000) + (derivative $\times$ 30)

 // CLAMP OUTPUT

 IF power_difference > maximum THEN

 power_difference $\leftarrow$ maximum

 ELSE IF power_difference < -maximum THEN

 power_difference $\leftarrow$ -maximum

 END IF

 Print power_difference to serial

 // CALCULATE MOTOR SPEEDS

 IF power_difference < 0 THEN

 speedLeft $\leftarrow$ maximum + power_difference

 speedRight $\leftarrow$ maximum

```

ELSE
    speedLeft ← maximum
    speedRight ← maximum - power_difference
END IF

// APPLY MOTOR SPEEDS
analogWrite(PWMA, speedLeft)
analogWrite(PWMB, speedRight)

END WHILE
END

```

2.3.4 Flowchart

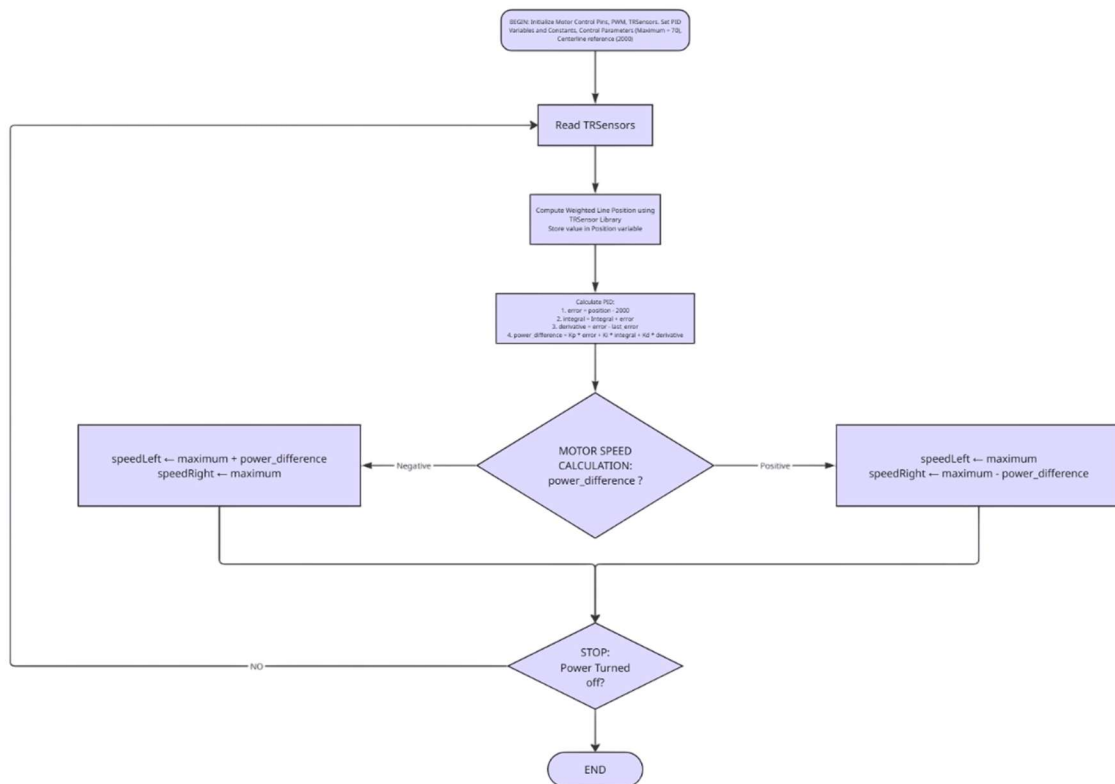


Figure 4: Flowchart

2.3.5 Code

This is the Line Following implementation, tested on the AlphaBot2-Ar platform. The complete source code is available in the GitHub repository <https://github.com/Misgar-Numan/Line-Following-Robot-PID>

```
/*
  LineFollowingRobotPID.ino
  An autonomous line-following robot using five IR sensors and PID
  control for smooth, stable line tracking. The robot follows a black
  line on a white surface with minimal oscillation and flickering
  compared to bang-bang control.
*/
#include "TRSensors.h"

#define NUM_SENSORS 5
TRSensors trs = TRSensors();
unsigned int sensorValues[NUM_SENSORS];
unsigned int last_proportional = 0;
long integral = 0;

#define PWMA 6    //Left Motor Speed pin (ENA)
#define AIN2 A0   //Motor-L forward (IN2).
#define AIN1 A1   //Motor-L backward (IN1)
#define PWMB 5    //Right Motor Speed pin (ENB)
#define BIN1 A2   //Motor-R forward (IN3)
#define BIN2 A3   //Motor-R backward (IN4)

void setup() {
  Serial.begin(9600);
  Serial.println("TRSensor example!!");
  pinMode(PWMA, OUTPUT);
  pinMode(AIN2, OUTPUT);
  pinMode(AIN1, OUTPUT);
  pinMode(PWMB, OUTPUT);
  pinMode(BIN1, OUTPUT);
  pinMode(BIN2, OUTPUT);
  analogWrite(PWMA, 0);
  analogWrite(PWMB, 0);
  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, HIGH);
  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, HIGH);

  // Make the calibration take about 10 seconds
  for (int i = 0; i < 400; i++) {
    trs.calibrate(); // reads all sensors 10 times
  }
  Serial.println("Calibration Done");
}
```

```
// Print the calibration minimum values measured when emitters
were on
for (int i = 0; i < NUM_SENSORS; i++) {
    Serial.print(trs.calibratedMin[i]);
    Serial.print(' ');
}
Serial.println();

// Print the calibration maximum values measured when emitters
were on
for (int i = 0; i < NUM_SENSORS; i++) {
    Serial.print(trs.calibratedMax[i]);
    Serial.print(' ');
}
Serial.println();
}

void loop() {
    // Get the position of the line.
    unsigned int position = trs.readLine(sensorValues);

    // The "proportional" term should be 0 when we are on the line.
    int proportional = (int)position - 2000;

    // Compute the derivative (change) and integral (sum) of the
    position.
    int derivative = proportional - last_proportional;
    integral += proportional;

    // Remember the last position.
    last_proportional = proportional;

    // Compute PID Output
    int power_difference = proportional / 5 + integral / 10000 +
    derivative * 30;

    // Compute the actual motor settings.
    const int maximum = 70;

    if (power_difference > maximum)
        power_difference = maximum;
    if (power_difference < -maximum)
```

```
    power_difference = -maximum;

    Serial.println(power_difference);
    if (power_difference < 0) {
        analogWrite(PWMA, maximum + power_difference);
        analogWrite(PWMB, maximum);
    } else {
        analogWrite(PWMA, maximum);
        analogWrite(PWMB, maximum - power_difference);
    }
}
```

3. Implementation

3.1 Build and Assembly

The Waveshare AlphaBot2-Ar mobile robot kit forms the foundation of this project. The kit features a snap-fit modular design, allowing for quick assembly of the chassis and electronic components without soldering. The AlphaBot2-Base consists of a pre-assembled circular chassis equipped with two N20 gear-motors, rubber wheels, necessary sensors, and a caster wheel. The AlphaBot2-Ar adapter board is mounted on top of the base using the supplied posts and standoffs, ensuring a stable connection between mechanical and control components. An Arduino UNO board fits directly into the adapter's Arduino-compatible header, aligning its pins with the interface. A 2×10-pin ribbon cable connects the adapter to the base, transmitting PWM control signals to the TB6612FNG motor driver and digital inputs from the IR and obstacle sensors. Power is supplied through two 14500 Li-ion cells housed in the base's battery holder, providing an operating voltage of approximately 7.4V. After assembly and power-up, the onboard OLED display and indicator LEDs confirm system initialization and readiness for sensor calibration.

3.2 Challenges Faced and How They Were Resolved

1. Custom Build Challenge

- *Challenges:*
 - IR sensors were spaced too far apart, reducing accuracy.
 - Uneven sensor mounting caused calibration errors.
 - Power fluctuations led to inconsistent sensor readings and unstable performance.
- *Solution:* The custom-built robot was replaced with the Waveshare AlphaBot2-Ar platform, which offered precise sensor alignment, stable power delivery, and reliable factory-calibrated components.

2. PID Gain Tuning

- *Challenge:* Determining suitable PID constants required repeated testing and tuning.
- *Solution:* Through iterative experimentation, optimal values were set as $K_p = 1/5$, $K_i = 1/10000$, and $K_d = 30$, achieving smooth and stable line tracking.

3. Calibration Sensitivity

- *Challenge:* Incomplete calibration exposure to black and white surfaces caused inaccurate sensor readings and unstable motion.
- *Solution:* Improved the calibration procedure to ensure each sensor detects both surfaces properly, enabling accurate line position estimation.

3.3 Assumptions

The implementation assumes the following conditions:

- The robot operates on a black line drawn on a white surface, ensuring maximum contrast for reliable IR detection.
- The line width is appropriate and matches the spacing of the IR sensor array, allowing accurate position sensing.
- The surface is flat and uniform, providing consistent infrared reflectance for stable sensor readings.
- The robot starts on or near the line before operation to enable proper initial detection and control response.
- A complete calibration process is performed, exposing each sensor to both black and white areas for accurate normalization.
- The ambient lighting conditions remain stable, minimizing interference with the IR sensors during line tracking.
- The line path consists of gradual curves, suitable for the robot's turning radius and PID control response.

4. Testing and Results

4.1 Tests Performed

To evaluate the performance and stability of the PID-controlled line-following robot, a series of controlled tests were conducted under different conditions:

1. **Straight Line Test:**
 - Verified the robot's ability to follow a straight black line on a white surface without oscillation.
 - Checked sensor readings and PID response for steady-state accuracy.
2. **Curved Path Test:**
 - Assessed the robot's performance on gradual and moderate curves.
 - Observed how effectively the PID controller managed differential motor speeds to maintain smooth turns.
3. **Calibration Accuracy Test:**
 - Evaluated the effectiveness of the 10-second calibration phase by analyzing sensor normalization values.
 - Verified consistent detection of black and white reflectance across all five sensors.
4. **Lighting Variation Test:**
 - Conducted under slightly different indoor lighting intensities to test robustness of IR sensing.

- Ensured minimal deviation in performance under stable but varied lighting conditions.

5. Speed Stability Test:

- Monitored the robot's movement at varying base speeds to ensure PID control maintained stability and prevented overshoot or oscillation.

4.2 Observations and Outcomes

- The robot successfully followed the black line with minimal lateral oscillations, demonstrating effective proportional control and stable feedback correction.
- On curved paths, the differential motor control enabled smooth and precise turning, with negligible delay in response.
- The PID tuning values ($K_p = 1/5$, $K_i = 1/10000$, $K_d = 30$) provided optimal performance, balancing responsiveness and stability.
- The sensor calibration process significantly improved line detection accuracy, reducing false readings and maintaining consistent tracking.
- The robot remained robust under normal indoor lighting, showing only minor variations in sensor readings without affecting trajectory.
- The AlphaBot2-Ar platform demonstrated reliable power management and hardware integration, contributing to smooth motor control and consistent behavior.

Overall, the robot achieved stable, oscillation-free navigation, validating the effectiveness of the PID control strategy over Bang-Bang control methods.

5. Conclusion and Future Scope

The implementation of a PID-controlled line-following robot on the AlphaBot2-Ar platform successfully demonstrated precise, smooth, and oscillation-free navigation along black lines on white surfaces. By utilizing a five-sensor IR array combined with carefully tuned PID gains, the robot was able to accurately detect line position and adjust motor speeds in real time, maintaining alignment even on curved paths. The calibration procedure ensured reliable sensor readings, and the hardware integration provided by the AlphaBot2-Ar platform offered consistent power supply and precise sensor positioning, which significantly contributed to the system's stability. Compared to Bang-Bang control, the PID approach minimized abrupt motor actions, reduced oscillations, and enabled more efficient energy usage, resulting in a highly responsive and reliable line-following system. Overall, this project validated the effectiveness of feedback-based control in autonomous mobile robotics and highlighted the advantages of using pre-integrated platforms like AlphaBot2-Ar for rapid prototyping and testing.

Looking forward, several enhancements can further extend the robot's capabilities. Incorporating dynamic obstacle detection and avoidance using ultrasonic or LiDAR sensors would enable navigation in more complex and unstructured environments. The system could be adapted to follow intricate paths with sharp turns and intersections, which would require adaptive PID tuning to handle sudden deviations efficiently. Integration with localization and mapping algorithms could transform the platform from a simple line follower to a fully autonomous mobile robot capable of navigating larger indoor spaces. Additionally, such robots could be deployed in industrial

warehouses, hospitals, or logistics centers to automate transport and delivery tasks, reducing human intervention and improving operational efficiency. Beyond industrial applications, the platform serves as an effective educational tool, providing hands-on experience with embedded systems, sensor calibration, control theory, and robotics programming, laying the foundation for developing advanced intelligent mobile systems in the future.

6. GitHub Repository

<https://github.com/Misgar-Numan/Line-Following-Localization-Robot>